

📌 Project Title:

"Lost & Found Management System with AI-Powered Image Matching"

💡 Concept:

A campus-wide or city-wide platform to **register, track, and claim lost items**, integrated with **image recognition** to match found items with lost reports automatically.

◆ Key Features:

1. **User Registration & Authentication** (Students, Staff, Admin)
 2. **Lost Item Reporting** – Upload details + image of the lost item.
 3. **Found Item Reporting** – Upload details + image of the found item.
 4. **AI-Powered Matching** – DB stores metadata & integrates a model to match images of lost/found items.
 5. **Status Tracking** – “Pending,” “Matched,” “Returned.”
 6. **Notification System** – Email/SMS alerts when a possible match is found.
 7. **Analytics Dashboard** – Category-wise, time-based reports.
 8. **Admin Tools** – Approve matches, remove false entries.
-

◆ DBMS Components:

- **Tables:** Users, Lost_Items, Found_Items, Matches, Notifications, Categories.
 - **Relationships:** One-to-many between Users and Items; Many-to-many between Lost_Items and Found_Items via Matches table.
 - **Indexes:** On item category, location, and AI-generated similarity score.
 - **Stored Procedures:** Auto-matching items on insert.
 - **Triggers:** Send notifications when a match is found.
-

◆ Tech Stack:

- **DB:** MySQL / PostgreSQL
- **Backend:** Java / Python Flask / Node.js
- **Frontend:** HTML, CSS, JS / React
- **AI Component:** TensorFlow / OpenCV for image similarity.

3 Features & Modules

User Side

- **Sign Up / Login** (Students, Staff, Guests).
- **Report Lost Item** (upload image, details, location, date).
- **Report Found Item** (upload image, details, location, date).
- **View Matches** – list of possible found items for your lost report.
- **Claim Process** – mark item as returned after verification.

Admin Side

- Approve or reject reports.
- View all lost/found items.
- Confirm matches suggested by AI.
- Manage users.
- Generate analytics reports (e.g., most common lost items, recovery rates).

7 Execution Flow

1. **User Registration** → store credentials in DB (hashed passwords).
2. **Report Lost/Found Item** → user uploads details + image.
3. **Store Data** in Lost_Items or Found_Items table.
4. **Trigger** runs → sends image to AI Matching Service.
5. **AI Model** computes similarity score between new entry and existing opposite category (Lost vs Found).
6. **Matches Table** updated with similarity scores.
7. **Notification Sent** to potential owner if score > threshold.

- 8. **Admin Verification** → confirms or rejects match.
 - 9. **Item Returned** → update status in DB.
-

8 DBMS Concepts Covered

- **Normalization** – avoid redundancy.
 - **Relationships** – one-to-many, many-to-many.
 - **Indexes** – on category, location, similarity_score for fast retrieval.
 - **Stored Procedures** – auto-run AI matching after insert.
 - **Triggers** – send notifications when match found.
 - **Transactions** – claim & update processes.
-

9 Dataset Strategy

- **Image Data:** COCO, ImageNet, Open Images Dataset.
- **Metadata:** Synthetic generation with Mockaroo or Faker.
- **Categories:** Wallets, Phones, Bags, Glasses, Keys, Watches, Documents.
- **Size:** Start with 500–1,000 entries for prototype.

X – Factor : adding **QR tagging, multimodal matching, and re-ranking**

QR Tagging & Quick Claim

Goal: Each item (especially found items) gets a unique **QR code** that links directly to its DB record.

- **Process:**
 1. When a found item is recorded, backend generates a **unique token** (UUID).
 2. Store it in Found_Items.qr_token.
 3. Use a library like **qrcode** (Python) or **qrcode-generator** (Node.js) to generate a QR image.

4. Print or display QR — stick it on the found item.
5. When scanned, QR opens a link like `https://yourapp.com/item/claim/{qr_token}`.
6. The owner can **verify** and **claim** directly.

Multimodal Matching (Text + Image)

Goal: Increase match accuracy by combining **AI image similarity** with **textual description matching**.

- **Image Matching:** Already handled by CNN embeddings.
- **Text Matching:**
 - Use **TF-IDF** or **Sentence Transformers** to create text embeddings from item descriptions, brand, and color.
 - Store these vectors in DB or FAISS.
- **Fusion:**
 - Final score = $w1 * \text{image_score} + w2 * \text{text_score}$ (weights adjustable).
 - Example: If location, date, and description match strongly but image match is moderate, the combined score can still be high enough.

Re-Ranking Matches by Metadata

Goal: Avoid mismatches when AI finds visually similar but irrelevant items.

- Factors to re-rank:
 1. **Location proximity** — closer locations get higher rank.
 2. **Date proximity** — matches within a few days get higher rank.
 3. **Color/Brand match** — exact matches get score boost.

Architecture Impact

Your updated **flow** will now be:

Upload Item → Compute Image Embedding → Compute Text Embedding → ANN Search (Image & Text separately) → Combine Scores → Apply

Metadata Re-ranking → Insert into Matches → Notify Owner → Generate QR if Found Item.

5 Tools & Libraries

- **QR Code Generation:** qrcode (Python) / qrcode-generator (Node.js).
- **Text Embeddings:** sentence-transformers (Python).
- **Vector Search:** FAISS for both image & text embeddings.
- **Metadata Matching:** Pure SQL for location/date, Python for score fusion.